

ALTERDUNE

Documentation Technique Complète

Version 4 — C++17 / SFML 3 / ImGui-SFML

Projet POO — ESILV — Second semestre 2025-2026

Table des matières

Table des matières	2
1. Vue d'ensemble du projet.....	4
1.1 Technologies utilisées	4
1.2 Structure des fichiers	4
2. Architecture orientée objet	5
2.1 Hiérarchie de classes	5
Entity — classe abstraite (entity.h).....	5
Player (player.h).....	5
Monster (monster.h).....	6
2.2 Polymorphisme — IA des monstres.....	6
3. Système d'actions (action.h)	7
3.1 Types d'actions.....	7
3.2 Application d'une action	7
3.3 Effets temporaires (tickEffects)	7
4. Items et inventaire.....	9
4.1 Structure d'un item (item.h).....	9
4.2 Slots d'équipement	9
4.3 Inventaire (inventory.h)	9
5. Classe Game — logique métier (game.h)	10
5.1 État principal.....	10
5.2 Chargement des données (loadData).....	10
5.3 Flux d'un run.....	10
5.4 Scaling des monstres	11
5.5 Détermination de la fin.....	11
6. Système Mercy	12
6.1 Conditions pour épargner	12
6.2 Équilibrage	12
7. Classes de personnage	13
8. Système de sauvegarde	14
8.1 Format du fichier.....	14
8.2 Reprise de partie	14
9. Interface graphique (Interface)	15
9.1 Écrans disponibles	15
9.2 Boucle principale	15
9.3 Système de fondu.....	15
10. Format des fichiers CSV	17
10.1 actions.csv.....	17
10.2 items.csv	17

10.3 monsters.csv	17
10.4 run.csv.....	17
11. Points clés pour la soutenance.....	18
11.1 Critères évalués et où les trouver	18
11.2 Fonctionnalités bonus.....	18

1. Vue d'ensemble du projet

Alterdune est un roguelike RPG au tour par tour développé en C++17. Le joueur traverse 10 combats consécutifs en choisissant entre deux voies : tuer les monstres (voie Conquête) ou les épargner via le système Mercy (voie Rédemption). Trois fins différentes sont possibles selon les choix effectués.

1.1 Technologies utilisées

Technologie	Rôle
C++17	Langage principal — POO, templates, std::optional, std::filesystem
SFML 3.0.2	Fenêtre, rendu, audio, gestion des textures/sprites
ImGui-SFML	Interface graphique immédiate (boutons, panels, onglets)
CSV	Données de jeu scriptées (actions, items, monstres, run)

1.2 Structure des fichiers

Fichier	Responsabilité
main.cpp	Point d'entrée — crée Game et Interface, lance la boucle
game.h / game.cpp	Logique métier complète (état, combat, sauvegarde)
interface.h / interface.cpp	Rendu ImGui/SFML, gestion des événements
entity.h / entity.cpp	Classe abstraite de base (joueur et monstres)
player.h / player.cpp	Classe Player — niveaux, équipement, XP
monster.h / monster.cpp	Classe Monster — IA, récompenses, scaling
action.h / action.cpp	Système d'effets (dégâts, buffs, mercy, durée)
item.h / item.cpp	Items consommables et équipables
inventory.h / inventory.cpp	Gestion des slots d'inventaire
Ressources/CSV/	actions.csv, items.csv, monsters.csv, run.csv
Save/save.txt	Sauvegarde persistante format clé=valeur

2. Architecture orientée objet

2.1 Hiérarchie de classes

Le projet repose sur une hiérarchie d'héritage centrée sur Entity, qui est une classe abstraite (elle possède une méthode virtuelle pure).

```
Entity (abstraite - playTurn() = 0)
/      \
Player  Monster
(contrôlé (IA : AGGRESSIVE
par l'UI) DEFENSIVE, RANDOM)
```

Entity — classe abstraite (entity.h)

Entity est la classe de base commune au joueur et aux monstres. Elle définit les attributs partagés et force les classes dérivées à implémenter playTurn().

```
class Entity {
public:
    std::string name;
    float hp, hpMax;
    float baseDommage, baseDefense;
    int mercy, mercyGoal;
    std::vector<Action> actions;
    std::vector<ActiveEffect> activeEffects;
    Inventory inventory;

    // Méthode virtuelle pure => Entity est abstraite
    virtual void playTurn(Entity* target,
        std::vector<std::string>* log = nullptr) = 0;

    void tickEffects(std::vector<std::string>* log = nullptr);
};
```

Player (player.h)

Player hérite d'Entity et ajoute : level, XP, gold, équipement par slots, pouvoirs de classe, gestion du level up différé.

```
class Player : public Entity {
public:
    int level, xp, gold;
    PowerClass powerClass; // BERSERKER / GUARDIAN / TRICKSTER
    std::vector<Action> powerActions;
    int pendingLevelUps = 0; // level ups en attente de choix
    EquipmentSlots equipment; // 9 slots d'équipement
    int nbKilled, nbSpared;

    void gainXP(int amount, std::vector<std::string>* log = nullptr);
    bool equipItem(Item* item, ...);
    void playTurn(Entity* target, ...) override {} // no-op
```

```
};
```

Monster (monster.h)

Monster hérite d'Entity et ajoute : xpReward, goldReward, catégorie (NORMIE/MINI_BOSS/BOSS), comportement IA, scaling de récompenses.

```
class Monster : public Entity {
public:
    int xpReward, goldReward;
    MonsterCategory category; // NORMIE / MINI_BOSS / BOSS
    MonsterIA ia;             // AGGRESSIVE / DEFENSIVE / RANDOM
    float rewardScale = 1.f;

    void playTurn(Entity* target, ...) override;
    void giveRewards(Player& player, ...);
};
```

2.2 Polymorphisme — IA des monstres

Le polymorphisme s'exprime dans Monster::playTurn() qui adapte le comportement selon le type d'IA :

```
void Monster::playTurn(Entity* target, ...) {
    int index = 0;
    switch (ia) {
        case MonsterIA::AGGRESSIVE:
            // Choisit l'action avec la valeur la plus haute
            for (int i = 1; i < actions.size(); i++)
                if (actions[i].value > actions[index].value) index = i;
            break;
        case MonsterIA::DEFENSIVE:
            // Préfère les actions selfTarget (buffs/soins)
            for (int i = 0; i < actions.size(); i++)
                if (actions[i].selfTarget) { index = i; break; }
            break;
        case MonsterIA::RANDOM:
            index = rand() % actions.size();
            break;
    }
    actions[index].apply(target, log);
}
```

3. Système d'actions (action.h)

Une Action est un effet applicable à une entité. Le système supporte 17 types différents, dont des effets instantanés, des effets sur la durée et des effets conditionnels.

3.1 Types d'actions

Type	Catégorie	Effet
DOMMAGE	Instantané	Inflige (value - baseDefense) HP
HEAL	Instantané	Restaure value HP (plafonné à hpMax)
BUFF_ATK / BUFF_DEF	Instantané	Augmente baseDommage ou baseDefense
DEBUFF_ATK / DEBUFF_DEF	Instantané	Réduit baseDommage ou baseDefense
MERCY_DOMMAGE	Instantané	Augmente la jauge mercy du monstre
MERCY_HEAL	Instantané	Réduit la jauge mercy du monstre
*_TIME (ex: DOMMAGE_TIME)	Durée 3 tours	Effet récurrent géré par tickEffects()
CONDITIONAL	Logique	Se déclenche si stat >= ou <= seuil

3.2 Application d'une action

La méthode apply() dispatche selon le type et modifie directement les attributs de la cible. selfTarget détermine si l'action s'applique à celui qui l'utilise ou à l'ennemi.

```
void Action::apply(Entity* target, std::vector<std::string>* log) {
    switch (type) {
        case ActionType::DOMMAGE: {
            float dmg = value - target->baseDefense;
            if (dmg < 0) dmg = 0;
            target->hp -= dmg;
            LOG(target->name + " prend " + to_string((int)dmg) + " degats");
            break;
        }
        case ActionType::BUFF_ATK_TIME:
            target->baseDommage += value;
            // Ajoute un ActiveEffect pour l'annuler après 3 tours
            target->activeEffects.push_back(
                {ActionType::BUFF_ATK_TIME, value, 3});
            break;
        // ... 15 autres cas
    }
}
```

3.3 Effets temporaires (tickEffects)

À chaque fin de tour, Entity::tickEffects() est appelée. Elle applique les effets actifs et supprime ceux dont la durée est épuisée, en annulant les modifications de stats.

```
void Entity::tickEffects(...) {
```

```
for (auto& e : activeEffects) {  
    // Appliquer l'effet ce tour  
    if (e.type == DOMMAGE_TIME) { hp -= e.value; }  
    e.duration--;  
}  
// Supprimer les effets expirés et annuler leurs buffs  
activeEffects.erase(remove_if(..., [](const ActiveEffect& e) {  
    if (e.duration <= 0) {  
        if (e.type == BUFF_ATK_TIME) baseDommage -= e.value;  
        return true;  
    }  
    return false;  
}), activeEffects.end());  
}
```


4. Items et inventaire

4.1 Structure d'un item (item.h)

Un Item possède un type (CONSUMABLE ou EQUIPPABLE), un slot cible pour l'équipement, et une liste d'actions à appliquer lors de l'utilisation ou de l'équipement.

```
class Item {
public:
    int id; string name, description;
    int value;           // prix en or
    ItemType type;      // CONSUMABLE ou EQUIPPABLE
    EquipSlot slotType; //
    HEAD/CHEST/MAIN_HAND/TWO_HANDS/LEGS/FEET/NECK/RING
    vector<Action*> actions;

    // user = joueur, enemy = cible optionnelle (monstres uniquement)
    bool use(Entity* user, vector<string*> log, Entity* enemy = nullptr);
    void applyEquipEffects(Entity* target, ...);
    void revertEquipEffects(Entity* target, ...);
};
```

4.2 Slots d'équipement

Le joueur dispose de 9 slots d'équipement. Les armes 1 main privilégient la main droite, les armes 2 mains bloquent les deux mains simultanément.

Slot	Quantité	Règle
HEAD	1	Casque
CHEST	1	Armure
MAIN_HAND	2	Main droite en priorité, gauche si droite occupée
TWO_HANDS	1	Occupe les deux mains simultanément
LEGS	1	Pantalon
FEET	1	Bottes
NECK	1	Collier
RING	3	Si tous pleins : choix de remplacement proposé

4.3 Inventaire (inventory.h)

L'inventaire stocke des InventorySlot (pointeur item + quantité). Les consommables sont limités à 5 slots. add() empile sur un slot existant ; use() appelle item->use() et retire si épuisé.

5. Classe Game — logique métier (game.h)

Game est le cœur du projet. Elle contient tout l'état du jeu et expose des méthodes que l'interface appelle. Elle ne fait aucun rendu — séparation stricte logique/affichage.

5.1 État principal

```
class Game {
public:
    GameScreen screen;           // écran actuel (MENU, COMBAT, etc.)
    optional<Player> currentPlayer;
    optional<Monster> currentMonster;
    int combatNumber;           // 1 à 10
    CombatPhase combatPhase;    // PLAYER_TURN / MONSTER_TURN / END
    CombatResult combatResult;  // NONE / VICTORY / SPARED / DEFEAT
    vector<Monster> pathChoices; // 1 ou 2 choix de chemin
    vector<string> combatLog;    // messages du tour en cours
    vector<Item*> lootPool, shopPool;
    GlobalStats globalStats;     // persistant entre runs
    vector<RunRecord> runHistory; // historique illimité
};
```

5.2 Chargement des données (loadData)

Les 4 CSV sont chargés au démarrage de façon indépendante. Un crash dans un CSV n'empêche pas les autres de se charger (safeLoad avec try/catch par fichier).

```
bool Game::loadData() {
    auto safeLoad = [&](auto fn, const char* path) {
        try { ok &= (this->*fn)(path); }
        catch (...) { /* CSV malformé — on continue */ }
    };
    safeLoad(&Game::loadActions,    "Ressources/CSV/actions.csv");
    safeLoad(&Game::loadItems,      "Ressources/CSV/items.csv");
    safeLoad(&Game::loadMonsters,   "Ressources/CSV/monsters.csv");
    safeLoad(&Game::loadRunEntries, "Ressources/CSV/run.csv");
    return ok;
}
```

5.3 Flux d'un run

Un run se déroule en séquence de 10 combats. Voici le flux complet :

- startRun() : crée le joueur, initialise combatNumber = 1
- buildPath(hard) : sélectionne 1 ou 2 monstres depuis run.csv avec scaling
- startCombat(index) : installe le monstre choisi, passe en COMBAT
- Combat tour par tour jusqu'à CombatPhase::END
- applyRewards() : XP/or, sauvegarde automatique
- Si dernier combat (10) : endRun() → END_RUN
- Sinon : retour à INTERMEDIATE pour le prochain choix

5.4 Scaling des monstres

Les monstres sont plus forts au fil des combats. La formule de scaling est la suivante :

```
// Voie facile
scale = 1.0 + (combatNumber - 1) * 0.05f

// Voie difficile (+50% de scaling supplémentaire)
scale = scaleEasy * 1.5f

// Application sur le monstre
monster.hp          *= scale;
monster.hpMax       *= scale;
monster.baseDamage *= scale;
monster.rewardScale = scale; // récompenses proportionnelles
```

5.5 Détermination de la fin

```
RunEnding Game::determineEnding() const {
    const Player& p = currentPlayer.value();
    if (p.nbSpared == 0) return RunEnding::CONQUETE; // tout tué
    if (p.nbKilled == 0) return RunEnding::REDEMPTION; // tout épargné
    return RunEnding::EQUILIBRE; // mix
}
```

6. Système Mercy

Chaque monstre possède une jauge Mercy (0 à mercyGoal). Le joueur peut l'augmenter via l'action APAISEMENT (universelle) ou via des actions de classe (HEX, MIND_LINK pour le Trickster).

6.1 Conditions pour épargner

- La mercy du monstre atteint mercyGoal
- Le joueur clique sur le bouton Epargner (option Mercy en combat)
- Résultat : CombatResult::SPARED — le monstre n'est pas tué
- Récompense : XP seulement (pas d'or), multiplié par rewardScale

6.2 Équilibrage

Les mercyGoal ont été calibrés pour que l'APAISEMENT (val=8) prenne environ le même nombre de tours que le kill direct. Le Trickster avec HEX (val=15) est environ 40% plus rapide sur la voie mercy.

Combat	Kill (tours)	Mercy Apaisement	Mercy HEX (Trickster)
C1-C2	~2.5t	~2.8t	~1.5t
C3-C7	~3t	~3.4t	~1.8t
C8-C9	~5-6t	~7-8t	~4t
C10	~10t	~11t	~6t

7. Classes de personnage

Le joueur choisit sa classe au début de chaque run. Chaque classe a 3 pouvoirs exclusifs utilisables une fois par combat.

Classe	HP	ATK	DEF	Pouvoirs (IDs 0-43)
Berserker	80	12	3	RAMPAGE (22 dmg), BERSERKER_FURY (ATK+10/3t), ARMOR_BREAK (DEF ennemi -6)
Guardian	120	6	10	FORTIFY (DEF+8), IRON_SKIN (DEF+7/3t), ENDURE (Regen 7HP/3t)
Trickster	90	7	5	HEX (Mercy+15), MIND_LINK (Mercy+8/3t), ENFEEBLE (ATK ennemi -5/3t)

8. Système de sauvegarde

La sauvegarde est automatique après chaque combat (dans `applyRewards()`). Le fichier `Save/save.txt` est au format clé=valeur avec des sections.

8.1 Format du fichier

```
[GLOBAL]
nbRuns=5
nbTotalKills=32
nbTotalSpared=18
nbConquete=2
nbRedemption=1
nbEquilibre=2

[BESTIARY]
bestiary=1,Froggit,1,0           ; id,nom,tué,épargné

[HISTORY]
run=3,CONQUETE,12,0,6,145,320,45,1234567890

[RUN]                               ; absent si pas de run en cours
name=Tav
combatNumber=4
hp=95
... (équipement, inventaire)
```

8.2 Reprise de partie

Au démarrage, `loadSave()` restaure l'état complet si une section `[RUN]` est présente. Le joueur reprend sur l'écran intermédiaire avec son inventaire et équipement intacts. Si `combatNumber` est hors [1-10], le save corrompu est supprimé automatiquement.

9. Interface graphique (Interface)

L'interface est entièrement dans `Interface`. Elle lit l'état de `Game` mais n'y écrit jamais directement — tout passe par les méthodes publiques de `Game`. Cette séparation stricte est le principe MVC appliqué au jeu.

9.1 Écrans disponibles

GameScreen	Description
LOADING	Chargement des CSV + attente d'input utilisateur
MENU	Menu principal avec logo, nouvelle partie, reprise, stats
CLASS_SELECT	Choix de classe avec cartes détaillées (stats + pouvoirs)
COMBAT	Combat tour par tour avec log coloré et effets visuels
INTERMEDIATE	Entre deux combats : loot, équipement, repos, marchand, chemin
END_RUN	Récapitulatif de fin (stats de combat + run + historique)
STAT_COLLECTION	Stats joueur, bestiaire, stats globales, historique scrollable
OPTIONS	Sliders volume musique et effets

9.2 Boucle principale

```
void Interface::run(Game& game) {
    initWindow();
    initImGui();
    initAssets();    // charge textures, fonts, audio

    while (window.isOpen()) {
        processEvents(game);
        float dt = deltaClock.restart().asSeconds();
        ImGui::SFML::Update(window, dt);
        update(game, dt); // animations, chargement, fade
        render(game);     // dispatch vers le bon écran
        window.clear();
        ImGui::SFML::Render(window);
        window.display();
    }
}
```

9.3 Système de fondu

Les transitions entre écrans utilisent un fondu noir (`FadeState : NONE/FADE_OUT/FADE_IN`). Un bloqueur d'input transparent est rendu pendant le fondu pour éviter les clics involontaires.

```
void Interface::startFade(GameScreen target) {
    if (fadeState != FadeState::NONE) return; // guard anti-double
    fadeTarget = target;
    fadeState = FadeState::FADE_OUT;
```

```
        fadeAlpha = 0.f;
    }

    // Dans update() :
    // FADE_OUT : alpha 0 -> 1, au pic : game.screen = fadeTarget
    // FADE_IN  : alpha 1 -> 0
```


10. Format des fichiers CSV

Tous les CSV utilisent le séparateur ";" et sont chargés au démarrage. Modifier ces fichiers permet de changer le contenu du jeu sans recompiler.

10.1 actions.csv

100 actions (ids 0-99). IDs 0-43 : actions joueur. IDs 60 : APAISEMENT (universel). IDs 61-99 : actions monstres uniquement (sans mercy).

```
id;name;description;type;value;selfTarget
0;RAMPAGE;Vous arrêtez de penser et commencez à frapper.;DOMMAGE;22;FALSE
28;HEX;Vous lancez un sortilège de charme.;MERCY_DOMMAGE;15;FALSE
60;APAISEMENT;Une approche calme et délibérée.;MERCY_DOMMAGE;8;FALSE
```

10.2 items.csv

100 items (ids 1-100). ~45 équipables répartis sur tous les slots, ~55 consommables.

```
id;name;description;value;type;actionIds;slotType
1;Hache de Fer;Une hache lourde et brutale.;15;EQUIPPABLE;1;MAIN_HAND
20;Potion de Soin;Restaure 20 PV instantanément.;10;CONSUMABLE;13;NONE
```

10.3 monsters.csv

100 monstres (ids 1-100). IDs 1-70 : NORMIE, 71-90 : MINIBOSS, 91-100 : BOSS. Les monstres n'utilisent que des actions non-mercy.

```
id;name;hp;baseDommage;baseDefense;mercy;mercyGoal;xpReward;goldReward;category;ia;actionIds;lootIds
1;Froggit;30;7;1;0;22;20;10;NORMAL;RANDOM;46|61;20|22
71;MimicBox;65;11;5;0;48;60;40;MINIBOSS;DEFENSIVE;62|66|82;2|6|8
91;QueenByte;100;15;6;0;80;150;100;BOSS;AGGRESSIVE;96|90|59|38;27|19|13
```

10.4 run.csv

Définit les pools de monstres pour chaque combat. fixedPool=1 pour combat 1 (pool unique) et combat 10 (boss). Les IDs sont séparés par "|".

```
combatId;fixedPool;easyMonsterIds;hardMonsterIds
1;1;1|2|3|4|5;
2;0;1|2|3|4|5|6|7|8|9|10;6|7|8|9|10|11|12
8;0;71|72|73|74|75|76;77|78|79|80|81
10;1;91|92|93|94|95;
```

11. Points clés pour la soutenance

11.1 Critères évalués et où les trouver

Critère (points)	Où dans le code
Héritage (2pt)	entity.h → player.h + monster.h (héritage public)
Classe abstraite (1pt)	entity.h : playTurn() = 0
Classes dérivées (1pt)	player.h override + monster.h override
Encapsulation (2pt)	player.h : nbKilled_/nbSpared_ privés + accesseurs
Polymorphisme (2pt)	monster.cpp : playTurn() switch(ia) — 3 comportements
Système ACT et Mercy (2pt)	action.cpp : MERCY_DOMMAGE/MERCY_HEAL + entity::tickEffects
Lecture fichiers (2pt)	game.cpp : loadData() + safeLoad + 4 CSV
Menu principal (3pt)	interface.cpp : renderMenu() + renderStatCollection()
Système de combat (3pt)	interface.cpp : renderCombat() + game.cpp : checkCombatEnd()
10 victoires + fins (2pt)	game.cpp : isLastCombat() + determineEnding()

11.2 Fonctionnalités bonus

- Interface graphique SFML + ImGui complète (vs console)
- Sauvegarde persistante avec reprise de partie
- 3 classes de personnage avec pouvoirs exclusifs
- Système d'équipement sur 9 slots
- Level up avec choix de stats
- Scaling progressif des monstres (5% par combat)
- Fondu noir entre scènes
- Bestiaire masqué (monstres non rencontrés = ???)
- Historique illimité des runs
- 100 actions, 100 items, 100 monstres scriptés en CSV